Next Session — Door Centring A/B

One change, same binary in both arms, battery as a hard gate

> **Where we stand after R1.** Gate 1 **PASSES** on the well-charged subset: battery ≥60% → 12/13 reached, A→B 7/7, B→A 5/6, median 65 s, median 0 collisions. **The golden baseline reproduces.** Below 50% battery the same binary drops to 5/12 and B→A to 1/6. Thirty runs, dry batch, five deliberate perturbation runs (displaced up to 2.07 m, facing up to 230° off) — **all five arrived**, and none of the nine failures was a perturbed run.

0. What this session tests, and why

Adrián's observation on the floor — "*on the way back it always drifts a little to one side before the door frame and puts itself in a worse position*" — is confirmed in the data and has a measured mechanism:

DISTANCE TO DOOR	2.2 M	1.4 M	1.0 M	0.7 M
Runs that failed — offset from the axis	−0.35	−0.19	−0.16	**−0.19 m**
Runs that crossed clean	−0.32	−0.08	−0.04	**−0.01 m**

Every run arrives ~0.35 m off-axis to the robot's **left**. Success depends entirely on whether it re-centres in the last metre and a half. The physical margin is only **±0.20 m** (0.99 m opening, 0.29 m effective half-width with arm swing), so arriving at 0.19 m is already at the limit.

The cause is not missing code. The strafe-to-axis servo already exists in the crossing phase, but it is gated to fire only above **0.14 m** of error and to stop correcting **0.35 m** before the opening. Failures cross that boundary still off-axis and can no longer correct.

> **Ruled out before writing any code** (2481 real samples): "trust the vision more to centre itself" cannot work. The door bearing correlates **+0.51** with the true geometric bearing — vision measures well — but **−0.00 with the lateral offset within 1.2 m**. Bearing tells you where to point, not where you are: at 0.7 m, aiming at the centre reads ~0° from any offset. The approach-bias variant (`Door_BiasPlus` , +0.12 m) was also tested in the twin and **rejected**: it made centring worse. Neither goes on the robot.

1. The experiment

Twelve B→A runs, alternating arms. **Both arms run the same binary** — they differ only in two environment variables, so there is no code confound at all.

ARM	VARIABLES	MEANING
base	none (defaults)	Exactly golden behaviour: 0.14 m dead zone, stops correcting 0.35 m out
tight	`G1_DOOR_CTR_TOL=0.07` `G1_DOOR_CTR_S=0.18`	Half the dead zone, keeps correcting twice as close

Twin evidence (12 interleaved B→A runs with calibrated noise, synthetic camera and DOOR-VIS): base → 5/6 reached, 3 collisions, |offset| 0.069 m · **tight → 6/6, zero collisions, |offset| 0.012 m**, no time cost. The two offset distributions do not overlap.

Honest caveat. The twin's baseline offset (0.069 m) is milder than the real one (0.19 m), so the twin proves the *mechanism*, not that the change prevents real collisions. That is exactly what tomorrow measures. Also: with n=6 per arm the twin could not resolve differences smaller than ~1 run or ~0.04 m — two identical configurations differed by that much. **The metric that discriminates is the lateral offset, not reached/failed.**

2. Pre-flight

2.1 Code — the branch for this session

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING && git fetch origin && git checkout golden-plus-cen
tring && git log --oneline -2
```

Expect `d70083e` on top of `019ec85 shakedown B->A con DOOR-VIS`. This branch is golden with **one** change: the two gates turned into variables. Verify it is really one change:

```
git diff --stat golden-doorvis
```

Must report a single file and ~9 added lines. Nothing else differs from the binary that produced the golden window.

2.2 The rest of the ritual — unchanged

1. **Battery ≥ 80% to start** (see §4 — this is now a hard gate).
2. GPUs free: `nvidia-smi`. Stop the local LLM service if it holds the cards.
3. Perception server, then `curl -s http://127.0.0.1:8008/health` → `ok: true`, MODE gpu.
4. Meta-reasoner: `cd meta-reasoner-2.0 && python3 -m unittest discover -q` → OK.
5. Phone ritual (pair from the app, Bluetooth on) → USB proxy → `reloccheck`. **Do not proceed with** `nube=0 pts`: the app must be on the map/SLAM screen. Healthy is a few hundred obstacle cells (golden median 233).
6. Marker in its own terminal, heartbeat visible in the run console.
7. Dry batch: empty cup in the hand, no apron.

3. The run loop

Alternate the arms run by run: base, tight, tight, base, base, tight… Twelve runs, all B→A — this is the direction with the problem, and using one direction doubles the statistical power for a given number of runs. Reposition the robot at B between runs.

Arm BASE (golden behaviour)

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING && source ~/g1env/bin/activate
G1_SETUP="cup=hand,apron=off,fill=dry" G1_ARM=CTR_BASE G1_META2=2 G1_M2_CFG=config_meta2_g1payloa
d.json G1_M2_STATE=meta2_state_ctr_base.json G1_M2_STATE_INIT=meta2_state_twin_explored.json G1_M2
_REALCAL=1 G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 G1_DOOR_VIS=1
G1_PERC=127.0.0.1:8008 python3 g1_goto.py gotoviz A
```

Arm TIGHT (the change)

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING && source ~/g1env/bin/activate
G1_DOOR_CTR_TOL=0.07 G1_DOOR_CTR_S=0.18 G1_SETUP="cup=hand,apron=off,fill=dry" G1_ARM=CTR_TIGHT G1
_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_STATE=meta2_state_ctr_tight.json G1_M2_STATE_
INIT=meta2_state_twin_explored.json G1_M2_REALCAL=1 G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_D
OORLIB=1 G1_M2_FRAGSPEED=1 G1_DOOR_VIS=1 G1_PERC=127.0.0.1:8008 python3 g1_goto.py gotoviz A
```

The two lines differ in exactly two variables and the arm tag. Nothing else changes between runs — not the cup, not the start pose, not the app, not the code. Do not add `G1_M2_ABORT_WIN`, `G1_M2_ABORT_PROG` or `G1_M2_HELP_S`: the golden window ran on the binary defaults (75 s / 0.4 m / 8 s) and passing larger values makes the supervisor twice as patient, inflating the arrival rate.

Per run, write down

Arrival or failure · time · collisions · **battery %** · and whether the robot visibly scraped or brushed the frame. The lateral offset comes out of the log afterwards; your eye on the frame contact is the part no sensor records.

4. Battery — now a hard gate

> Yesterday's session ran 79% → 29% without recharging and the same binary went from **92%** arrivals above 60% battery to **42%** below 50%, with achieved cruise speed dropping 11% at an identical commanded speed. The meta-reasoner detected it without being told (tension ×2.75, fulfilment ÷5) while sensing reliability stayed flat — it degraded in **mobility**, not perception.
>
> **Rule from now on: start ≥80%, stop the batch at 60%, recharge, resume.** Log the battery on every run. A batch that crosses 60% is two populations, not one.

Optional, if the day allows (R1b — the causality check). After the twelve runs, deliberately continue to <45% with four more runs, then recharge above 80% and repeat four. If performance recovers, battery is causal; if it does not, the degradation is cumulative session effect (thermal, gait drift). One hour, and it settles a variable that affects every future campaign.

5. Acceptance criteria

SIGNAL	WHAT WOULD CONFIRM THE CHANGE
Lateral offset at the threshold (primary)	tight clearly tighter than base — twin showed 0.012 vs 0.069 m with no overlap. This is the mechanism metric and it discriminates far better than arrivals

Arrivals B→A	base ≈ 5/6 (the R1 rate), tight ≥ 5/6. With n=6 per arm this alone cannot decide — read it together with the offset
Collisions	fewer in tight, and none at the door frame
Time	no penalty expected (twin: 102 vs 106 s)
What would refute it	tight offsets as wide as base → the real drift is generated after the correction window closes, and the fix is aimed at the wrong place

Stop rule unchanged: three consecutive failures of the same mode → stop, data home, analyse offline. No code edits at the lab.

6. End of session

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING && git add -A && git commit -m "door centring A/B s
ession" && git push origin golden-plus-centring
```

Then tell me and I run the offset analysis per arm. If the change holds on the robot, the natural next step is folding it into `main` and moving on to R2 (water at 200 g on the same binary), with battery controlled from the start.

> **Not in this session, on purpose:** the approach-bias variant (tested and rejected), the meta state machine branch (its own gated trial, after this), water (next batch), and the B-pocket — which yesterday produced a second, independent failure mode: runs that entered the door well centred and got stuck *after* it. That one is a physical waypoint problem and deserves its own fix, not a control parameter.